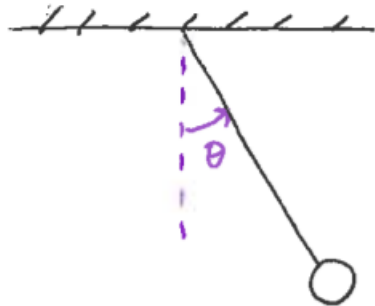


Problem 1

Using a simple set of ODEs for a non-linear pendulum, solve the system numerically



FBD:



Method 1

As simple as possible, without ODE45

```
% non-linear pendulum, simple function
clear all
```

```
% SYSTEM PARAMETERS
g = 9.81; % gravity
L = 1; % length
```

system
parameters

```
% TIME
tf = 4*pi*sqrt(L/g);
n = 4000;
h = tf/n; % time step
```

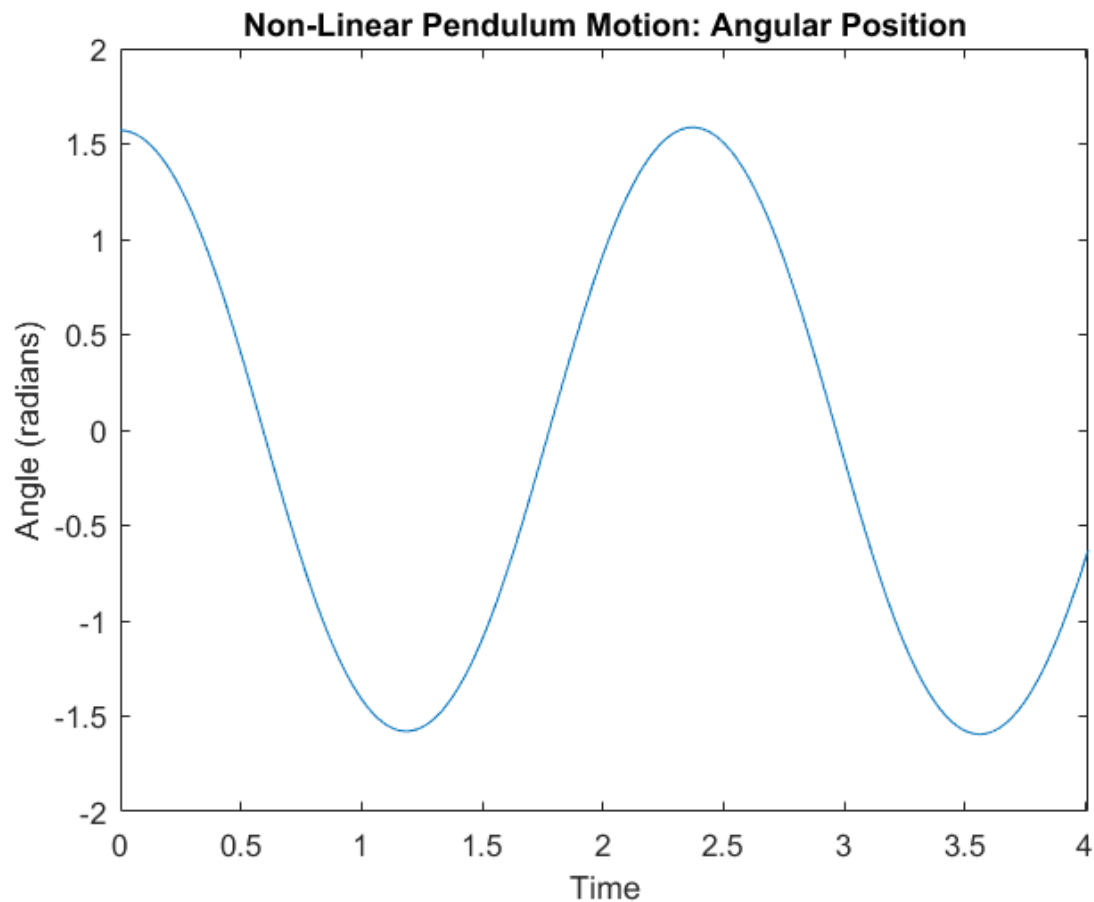
time

```
% INITIAL CONDITIONS
omega = 0; % angular velocity
theta = pi/2; % angle from the vertical
z = [ ];
```

initial
conditions

```
for i=1:n % Step the ODE forward in time.  
  
    a = -(g/L)*sin(theta); % angular acceleration  
    theta = theta + h*omega; % update position  
    omega = omega + h*a; % update velocity  
    z = [z theta]; % combine results  
  
end  
  
% FIGURES  
figure(1)  
plot(h:h:tf,z);  
title('Non-Linear Pendulum Motion: Angular Position')  
ylabel('Angle (radians)')  
xlabel('Time')  
xlim([0 tf])
```

Numerical
solution



Method 2

Use an Euler solver

```
clear all
% SYSTEM PARAMETERS
p.g = 9.81; % gravity
p.L = 1; % length

% TIME
tspan = [0 4*pi*sqrt(p.L/p.g)];
n = 4000;

% INITIAL CONDITIONS
omega0 = 0; % angular velocity
theta0 = pi/2; % angle from the verticle
cond = [theta0 omega0]; % [position velocity]

[tarray,z] = eulermethod(@RHS,tspan,cond,n,p);
theta = z(:,1);
omega = z(:,2);

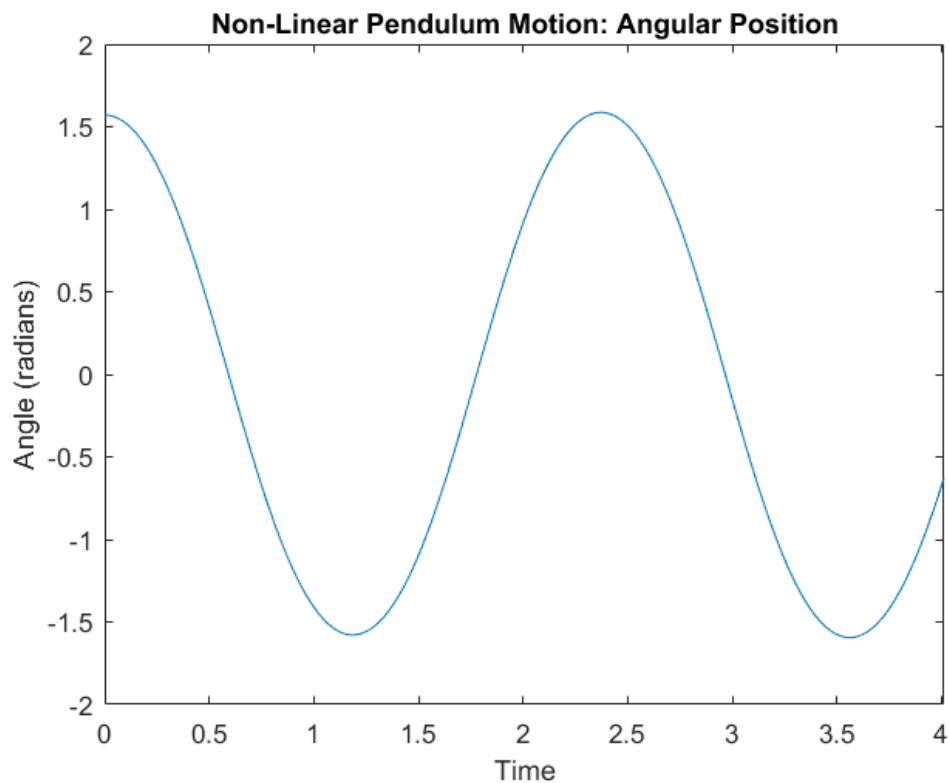
% FIGURES
figure(2)
plot(tarray,theta);
title('Non-Linear Pendulum Motion: Angular Position')
ylabel('Angle (radians)')
xlabel('Time')
xlim([0 tspan(2)])
```

System
parameters

time

initial
conditions

Numerical
solution



Method 3

Use ODE45

```
% SYSTEM PARAMETERS
```

```
g = 9.81; % gravity
```

```
L = 1; % length
```

System
parameters

```
% TIME
```

```
tf = 4*pi*sqrt(L/g);
```

```
n = 4000; h = tf/n; % time step
```

```
tspan = [h:h:tf];
```

time

```
% INITIAL CONDITIONS
```

```
omega0 = 0; % angular velocity
```

```
theta0 = pi/2; % angle from the vertical
```

```
cond = [theta0 omega0];
```

initial
conditions

```
% ODE45
```

```
system = @(t,z)[z(2); -g * sin(z(1))/ L];
```

```
[t,y] = ode45(system, tspan, cond);
```

```
theta = y(:,1); omega = y(:,2);
```

Numerical
solution

```
% FIGURES
```

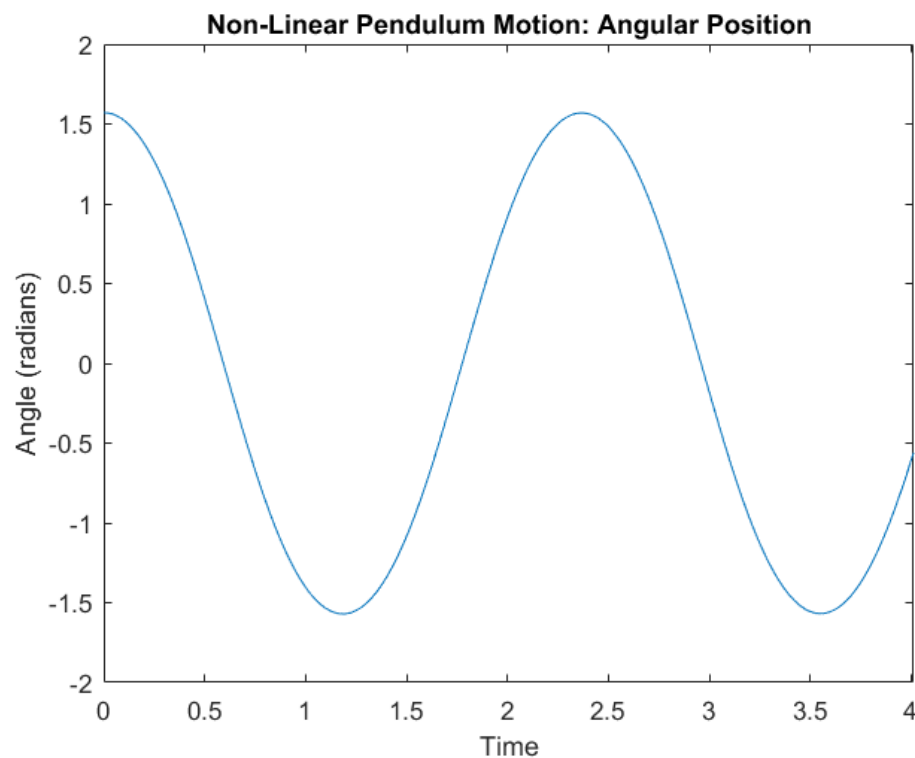
```
figure(3)
```

```
plot(t,theta)
```

```
title('Non-Linear Pendulum Motion: Angular Position')
```

```
ylabel('Angle (radians)')
```

```
xlabel('Time')
```



Effects of Time Step

Using code from Method 3, modify the time step and compare results

```
% Using modified code from part b
clear all

% SYSTEM PARAMETERS
p.g = 9.81; % gravity
p.L = 1; % length

% TIME
tspan = [0 4*pi*sqrt(p.L/p.g)];
n = [10000 5000 1000 250 100]; % changing step size

% INITIAL CONDITIONS
omega0 = 0; % angular velocity
theta0 = pi/2; % angle from the vertical
cond = [theta0 omega0]; % [position velocity]

for i = 1:length(n)
    h(i) = tspan(2)./n(i);
    [tarray,z] = eulermethod(@RHS,tspan,cond,n(i),p);
    theta = z(:,1);
    omega = z(:,2);

    %take smallest time step to be 'true' value
    if i == 1
        t_true = tarray;
        theta_true = z(:,1);
        omega_true = z(:,2);
        % compare results from other time steps
    else
        for j = 1:length(tarray)
            theta_true1 = interp1(t_true,theta_true,tarray(j));
            %interpolate true value from true solution
            Err(j) = abs(theta_true1-theta(j));
        end
        ErrSum(i) = sum(Err)/length(Err); %total error for given step
    end
end

size

% FIGURES
figure(4)
subplot(2,1,1)
plot(tarray,z(:,1));
hold on
title('Non-Linear Pendulum Motion: Angular Position')
ylabel('Angle (radians)')
xlabel('Time')
xlim([0 tspan(2)])
```

System parameters

time

initial conditions

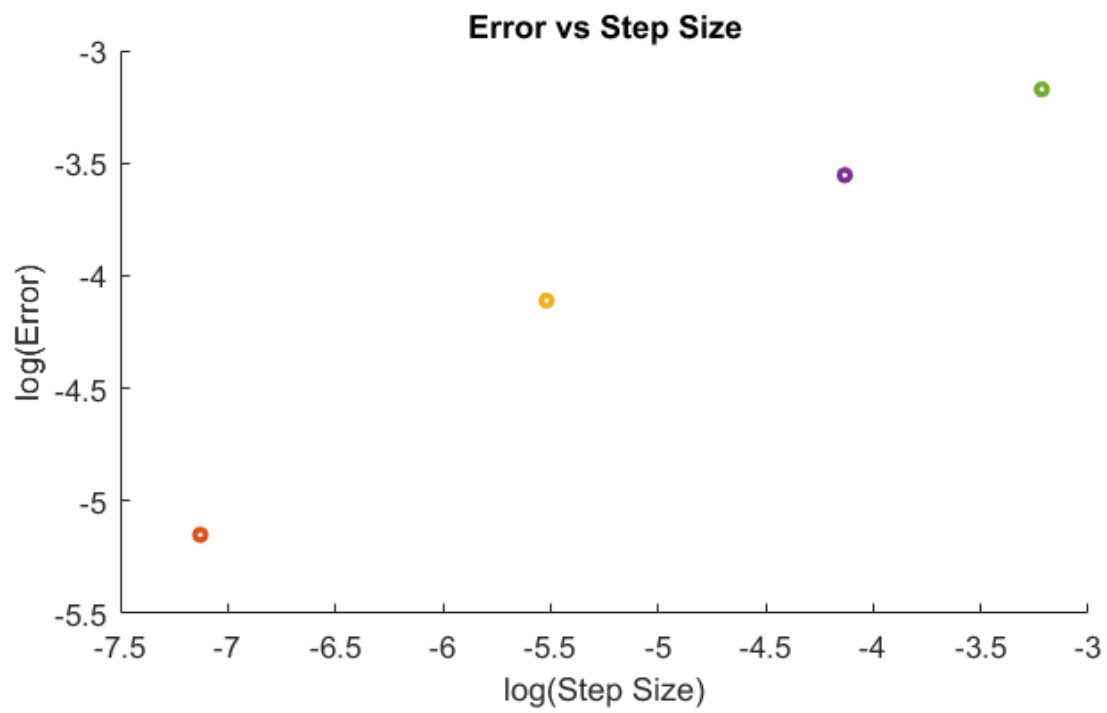
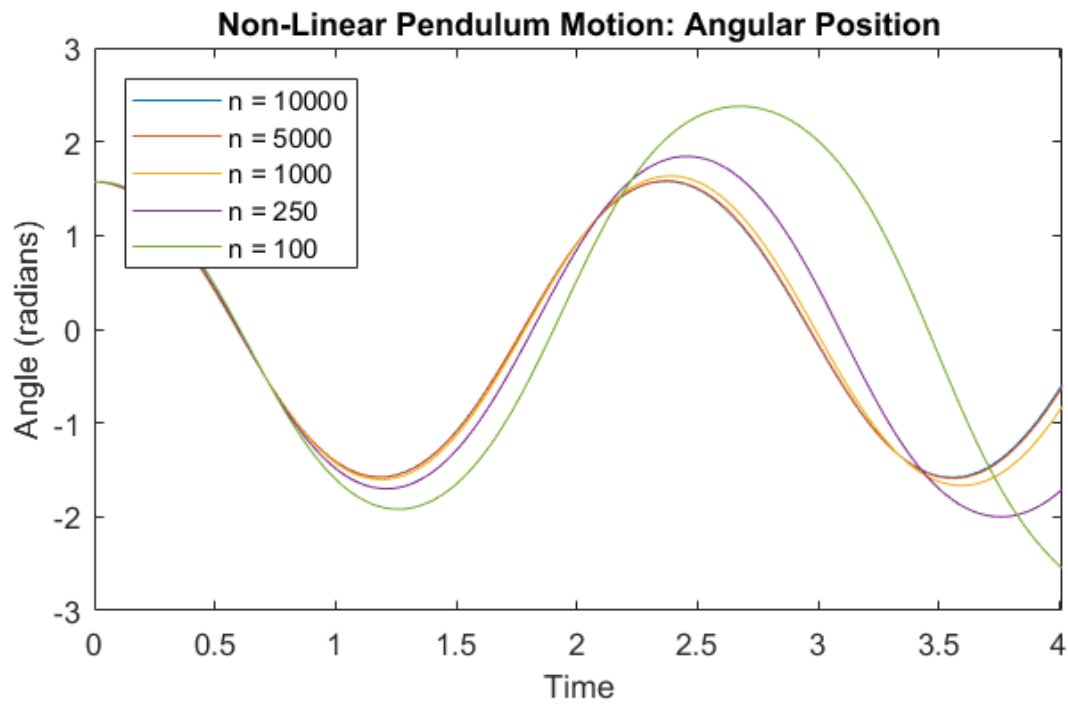
Numerical solution

```
end

legend(['n = ', num2str(n(1))], ['n = ', num2str(n(2))], ...
       ['n = ', num2str(n(3))], ['n = ', num2str(n(4))], ...
       ['n = ', num2str(n(5))], 'Location', 'northwest')
Color = get(gca, 'ColorOrder');

subplot(2,1,2)

scatter(log(h), log(ErrSum), 10, Color([1:5], :), 'LineWidth', 2, 'SizeData',
13) %Error growth is exponential
title('Error vs Step Size')
ylabel('log(Error)')
xlabel('log(Step Size)')
```



RelTol and AbsTol

Use RelTol and AbsTol with ODE45

```
clear all

% SYSTEM PARAMETERS
g = 9.81; % gravity
L = 1; % length

% TIME
tf = 4*pi*sqrt(L/g);
n = 4000;
h = tf/n;
tspan = [h:h:tf];

% INITIAL CONDITIONS
omega0 = 0; % angular velocity
theta0 = pi/2; % angle from the vertical
cond = [theta0 omega0];

% TOLERANCES
field1 = 'RelTol'; value1 = {1e-3, 1e-4, 1e-5};
field2 = 'AbsTol'; value2 = {1e-3, 1e-4, 1e-5};
opts = struct(field1,value1,field2,value2);

% ODE45
system = @(t,z)[z(2); -g * sin(z(1))/ L];

for i = 1:length(opts)
    [t,y] = ode45(system, tspan, cond,opts(i));

    % assuming largest error occurs at latest time
    % at latest time, pendulum should be back at initial position pi/2
    RelTol2(i) = abs(y(end,1)-pi/2)/min(abs(y(end,1)), abs(pi/2));
    AbsTol2(i) = abs(y(end,1)-pi/2);

    % FIGURES
    figure(5)
    plot(t,y(:,1))
    hold on
    title('Non-Linear Pendulum Motion: Angular Position')
    ylabel('Angle (radians)')
    xlabel('Time')
    xlim([2.35 2.38])
end

% FIGURES
figure(5)
plot([0 tf],[pi/2 pi/2],'--')
```

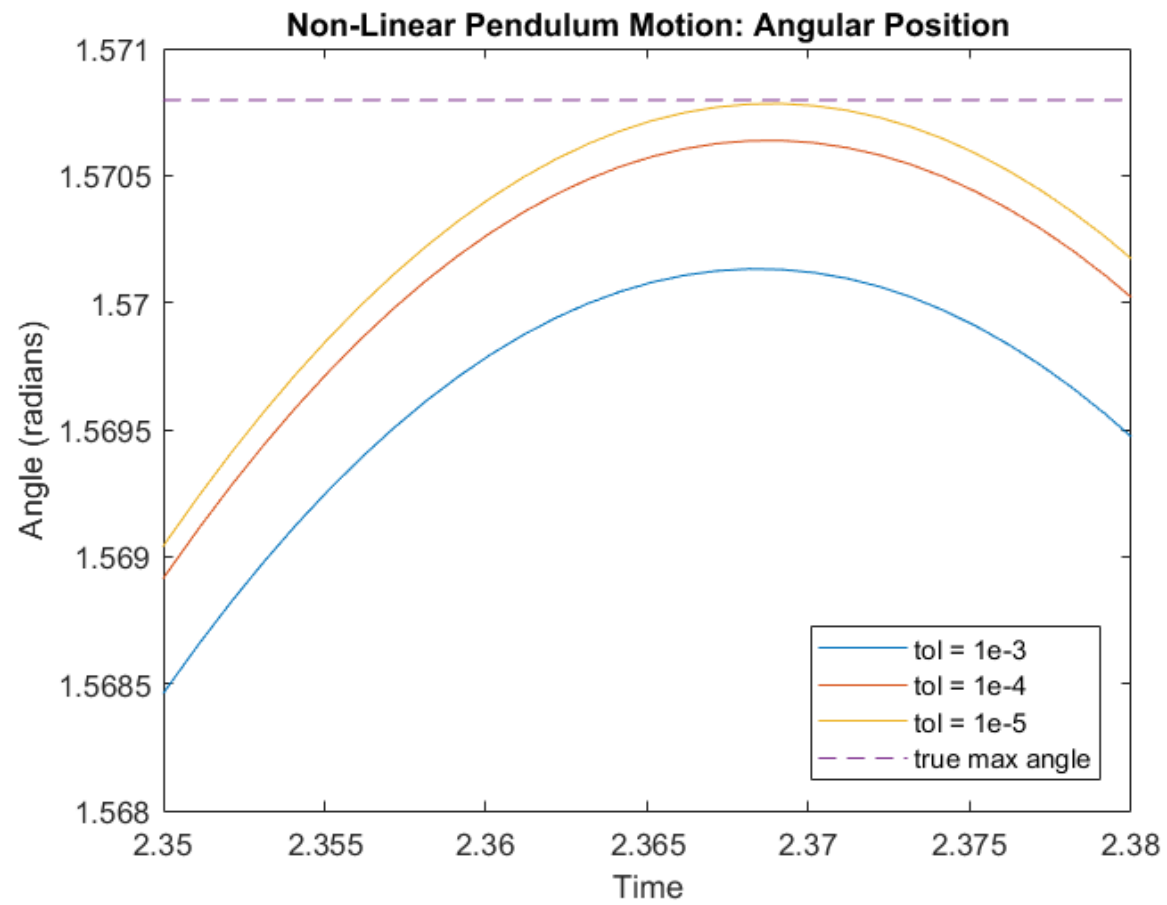
System parameters

time

initial conditions

Numerical solution


```
legend('tol = 1e-3', 'tol = 1e-4', 'tol = 1e-5', 'true max angle', ...  
      'Location', 'southeast')
```



Function: eulermethod

```

function [tarray, zarray] =
    eulermethod(odefun,tspan,z0,n,p)

%
% INPUTS:
%   odefun - function used to calculate new
position/velocity
%   tspan - array with initial and final time
%   z0 - initial conditions
%   n - number of steps
%   p - paramters used for odefun
%
% OUTPUTS:
%   tarray - array of time values
%   zarray - matrix of position and velocity at time values
in tarray

    tarray = linspace(tspan(1),tspan(2),n+1); % initialize
tarray

    h = tarray(2)-tarray(1); % size of time step

    zarray = zeros(n+1,length(z0));
    zarray(1,:) = z0'; % initialize zarray

    for i=1:n % Step through the time interval
        z = zarray(i,:);
        t = tarray(i);
        znew = z + h*odefun(z,p);
        zarray(i+1,:) = znew';
    end
end
end

```

Function: RHS

```
function zdot = RHS(condition,parameter)
% INPUTS:
%   condition = [position velocity]
%   parameter.g = gravity
%   parameter.L = length
%
% OUTPUT:
%   zdot = [velocity acceleration]'

x = condition(1);
v = condition(2);

a = -(parameter.g/parameter.L)*sin(x);

zdot = [v,a];

end
```